Graph Theory and Algorithms

Ph.D. Course – Marco Viviani

Graph Compression, Summarization, and Embedding
(May 15, 2025 / 15:00-17:00)

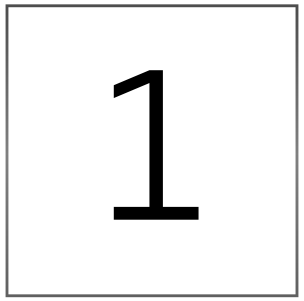
TABLE OF CONTENTS

1. SCENARIO
   Intro, Basic Notions, and Open Issues

2. GRAPH COMPRESSION
   Intro and Some Compression Models

3. GRAPH SUMMARIZATION
   Intro and Taxonomy of Approaches

4. GRAPH EMBEDDING
   An Overview



Scenario

Intro, Basic Notions, and
Open Issues

Graph sizes in 2018

Graph	IVI	IEI (symmetrized)
com-Orkut	3M	234M
Twitter	41M	1.46B
Friendster	124M	3.61B
Hyperlink2012-Host	101M	2.04B
Facebook (2011) [1]	721M	68.4B
Hyperlink2014 [2]	1.7B	124B
Hyperlink2012 [2]	3.5B	225B
Facebook (2018)	> 2B	> 300B
Google (2018)	?	?

Publicly available graphs
Private graphs

[1] The Anatomy of the Facebook Social Graph, Ugander et al. 2011
[2] http://webdatacommons.org/hyperlinkgraph/

From: Graph Compression – Data compression (Lecture 5)

Graph compression in industry

NetflixGraph Metadata Library: An Optimization Case Study

by Drew Koszewnik

Problem: running into memory issues when storing the movie property graph in memory

Solution: Compact Encoded Data Representation

We knew that we could hold the same data in a more memory-efficient way. We created a library to represent directed-graph data, which we could then overlay with the specific schema we needed.

Results

When we dropped this new data structure in the existing NetflixGraph library, our memory footprint was reduced by 90%. A histogram of our test application from above, loading the exact same set of data, now looks like the following:

Source: Netflix Tech Blog

From: Graph Compression – Data compression (Lecture 5)

Graph compression in industry

Compressing Graphs and Indexes with Recursive Graph Bisection

Abstract

Graph reordering is a powerful technique to increase the locality of the representations of graphs, which can be helpful in several applications. We study how the technique can be used to improve compression of graphs and inverted indexes.

Our experiments show a significant improvement of the compression rate of graph and indexes over existing heuristics. The new method is relatively simple and allows efficient parallel and distributed implementations, which is demonstrated on graphs with billions of vertices and hundreds of billions of edges.

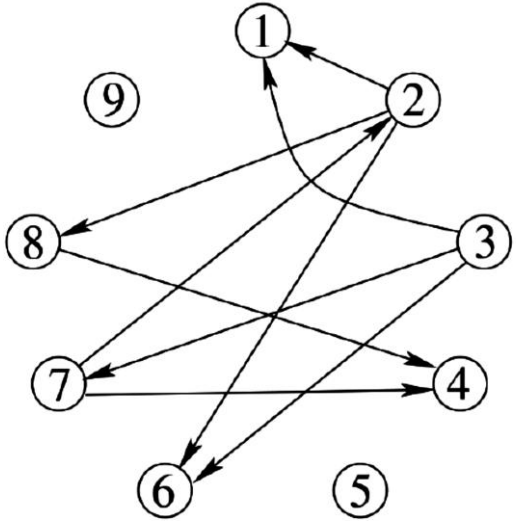
Source: Facebook Research

Operations on Graphs

- **Static graphs**:
  - Scanning the whole graph (i.e., the storage cost),
  - `get_neighbors(v)` (in/out neighbors for digraphs),
  - `is_edge(u,v)` (is the (u, v) edge present in G?).
- **Dynamic graphs**:
  - Insert/delete nodes/edges.

From: Compressed Sparse Row Format for Representing Graphs

Graph Representations



(a) example graph

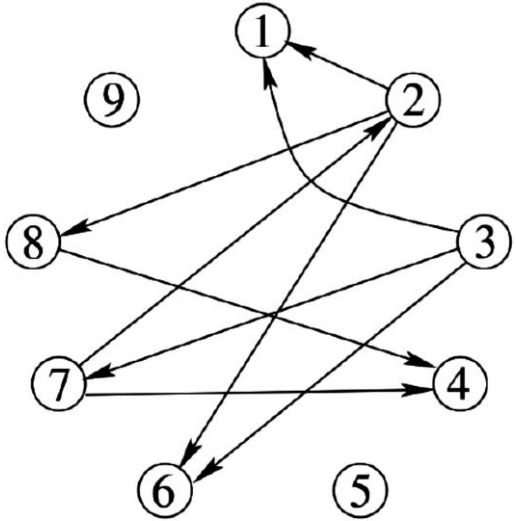
- **Edge List**

2	1
2	6
2	8
3	1
3	6
3	7
7	2
7	4
8	4

(b) edge list

From: Compressed Sparse Row Format for Representing Graphs

Graph Representations … Cont'd



(a) example graph

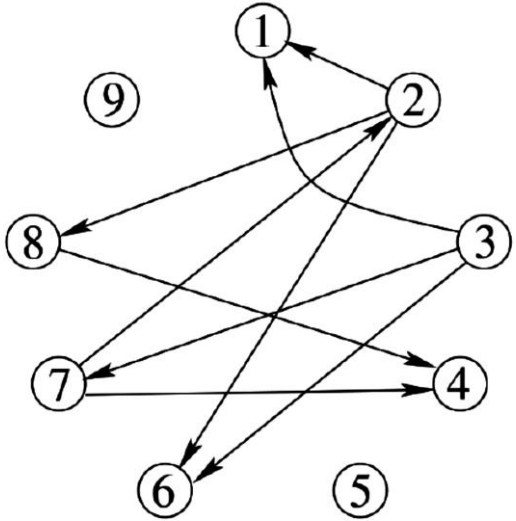
- **Adjacency Matrix**
  - Vertices labeled from 0 to $n-1$.
  - Entry of "1" if edge exists, "0" o.w. (or the weight on the edge).

	1	2	3	4	5	6	7	8	9
1									
2	1					1		1	
3	1					1	1		
4									
5									
6									
7		1		1					
8				1					
9									

(c) adjacency matrix

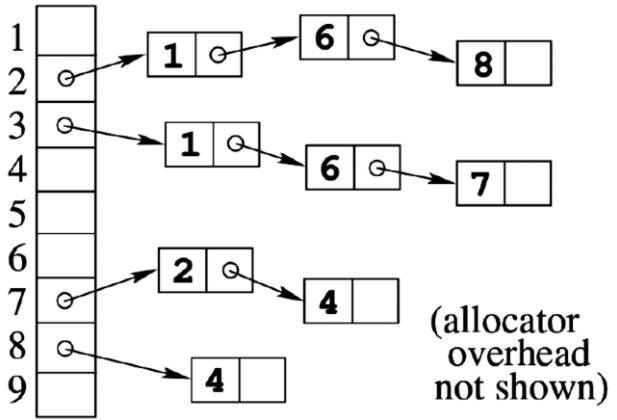
From: Compressed Sparse Row Format for Representing Graphs

Graph Representations … Cont'd



(a) example graph

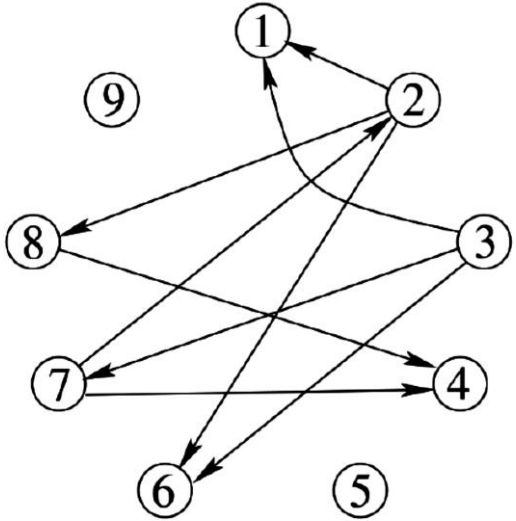
- **Adjacency List**
  - Array of pointers (one per vertex).
  - Each vertex points to a list of its neighbors.



(d) adjacency lists

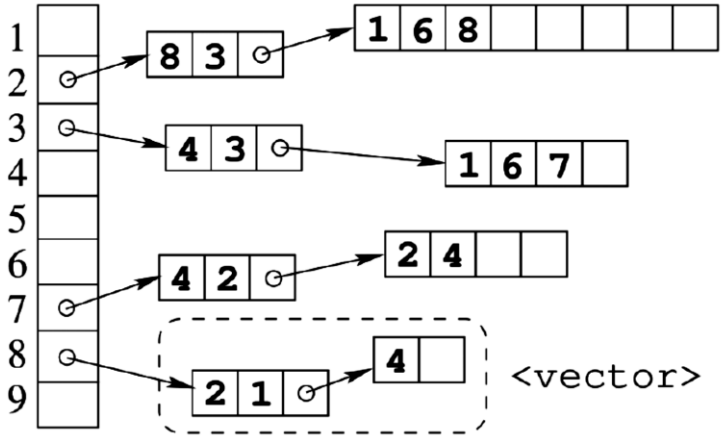
From: Compressed Sparse Row Format for Representing Graphs

Graph Representations … Cont'd



(a) example graph

- **Adjacency Vectors**
  - An array indexed by "from" vertexID contains entry points to `<vector>`s of "to" vertexIDs.



(e) adjacency vectors

A partially filled data array, which contains the capacity of the data array, the number of positions in the array occupied by user data, and a pointer to the data array itself

From: Graph Compression – Data compression (Lecture 5)

Computational Costs – Time

Operation	Adjacency Matrix	Edge List	Adjacency List
scan_graph	$O(n^2)$	$O(m)$	$O(m+n)$
get_neighbors	$O(n)$	$O(m)$	$O(d)$
is_edge	$O(1)$	$O(m)$	$O(d)$
insert edge	$O(1)$	$O(1)$	$O(1)$ or $O(d)$
delete edge	$O(1)$	$O(m)$	$O(d)$

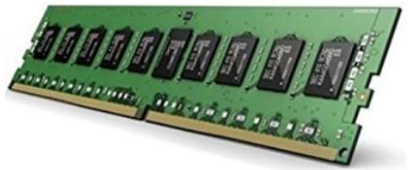
From: Graph Compression – Data compression (Lecture 5)

Computational Costs – Space

Hyperlink2012 Graph

- n = 3.6B, m = 225B (undirected edges)
- Vertex ids fit into 4 bytes
- > 900Gb to store in CSR format

We are going to detail it later



32Gb DRAM: about 300$*

So, about 9000$ of memory just to store the graph.
Doesn't include memory needed to run algorithms on it!

*Source: Hynix HMA84GR7MFR4N-UH 32GB DDR4-2400 ECC REG DIMM Server Memory

Compression VS Summarization

- **Graph compression** applies various encoding techniques, so the resultant graph needs less **storage space**.

  → **Result**: a more compact representation (e.g., through encoding, compressed adjacency lists, etc.)

- **Graph summarization** aggregates nodes having similar structural properties/patterns to represent a graph with reduced main **memory requirements**.

  → **Result**: a smaller graph or a summarizing model.

Seo, H., Park, K., Han, Y., Kim, H., Umair, M., Khan, K. U., & Lee, Y. K. (2018). An effective graph summarization and compression technique for a large-scaled graph. The Journal of Supercomputing, 1-15

And… What about Graph Embedding?

- The goal of **graph embedding** is to represent nodes, entire graphs, or subgraphs in low-dimensional vector spaces, while preserving as much as possible the structural and/or semantic properties of the original graph.

- This transformation enables more efficient analysis, comparison, and use of graph data in downstream tasks such as classification, clustering, and link prediction.

→ **Result**: a set of vectors, not necessarily a compressed graph, but a more compact representation.

2

Graph Compression

Intro and Some
Compression Models

Graph Compression

- Aim: **storage-efficient processing of large graphs**
  - This is becoming increasingly important w.r.t. Big Data Analysis
- Many different **areas of application**:
  - Web graphs
  - Biology networks
  - Social graphs
  - …
- Many of these approaches originated in the area of **data compression** and **high-performance scientific computing**.

The Compressed Sparse Row (CSR) Representation

- The **Compressed Sparse Row (CSR)** Representation originated in high-performance scientific computing as a way to represent **sparse matrices**, whose rows contain mostly zeros.

- "Old" representation
  - Appeared in the mid-60.

- The basic idea is to pack the column indices of non-zero entries into a dense array → How?

The Compressed Sparse Row (CSR) Representation … How? (1)

- The Compressed Sparse Row represents an mxn matrix M by **three (one-dimensional) arrays**, that respectively contain non-zero values, the row pointers, and column indices.
- The arrays V and C_{Index} are of length nnz, and contain the **non-zero values** and the **column indices** of those values respectively.
  - nnz denotes the number of nonzero entries in M.
- The array R_{Index} **encodes** the index in V and C_{Index} where the given row starts. Its length is $m + 1$.

The Compressed Sparse Row (CSR) Representation … How? (2)

- The R_{Index} vector stores the **cumulative number of non-zero elements** upto (not including) the i-th row.

- It is defined by the **recursive relation**:
  - $R_{Index}[0] = 0$.
  - $R_{Index}[i] = R_{Index}[i-1]$ + number of non-zero elements in the $(i-1)$th row of the matrix.

- To find the number of non-zero elements in say row i, we perform:
  - $R_{Index}[i+1] - R_{Index}[i]$.

The Compressed Sparse Row (CSR) Representation – Example 1 (1)

For **example**, the matrix

0	0	0	0
5	8	0	0
0	0	3	0
0	4	0	0

is a 4×4 matrix with 4 nonzero elements, hence:

- $V = \quad (5 \;\; 8 \;\; 3 \;\; 4)$
- $C_{Index} = (0 \;\; 1 \;\; 2 \;\; 1)$

- What about the R_{Index}?

The Compressed Sparse Row (CSR) Representation – Example 1 (2)

For **example**, the matrix

0	0	0	0
5	8	0	0
0	0	3	0
0	4	0	0

- $R_{Index}[0] = 0$
- $R_{Index}[1] = R_{Index}[0]$ + n. of non-zero elements in row 0, i.e, 0 + 0 = 0.
- $R_{Index}[2] = R_{Index}[1] + 2 = 2$
- $R_{Index}[3] = R_{Index}[2] + 1 = 3$
- $R_{Index}[4] = R_{Index}[3] + 1 = 4$
- Therefore, $R_{Index} = (0\ 0\ 2\ 3\ 4)$

The Compressed Sparse Row (CSR) Representation – Example 1 (4)

From the following **CSR representation** to the matrix:

- $V = \quad (5 \;\; 8 \;\; 3 \;\; 4)$
- $C_{Index} = (0 \;\; 1 \;\; 2 \;\; 1)$
- $R_{Index} = (0 \;\; 0 \;\; 2 \;\; 3 \;\; 4)$

- Row 0 = 0 – 0 elements (empty row)
- Row 1 = 2 – 0 = 2 elements (in column 0 and 1)
- Row 2 = 3 – 2 = 1 elements (in column 2)
- Row 3 = 4 – 3 = 1 elements (in column 1)

0	0	0	0
5	8	0	0
0	0	3	0
0	4	0	0

The Compressed Sparse Row (CSR) Representation – Example 1 (3)

For **example**, the matrix

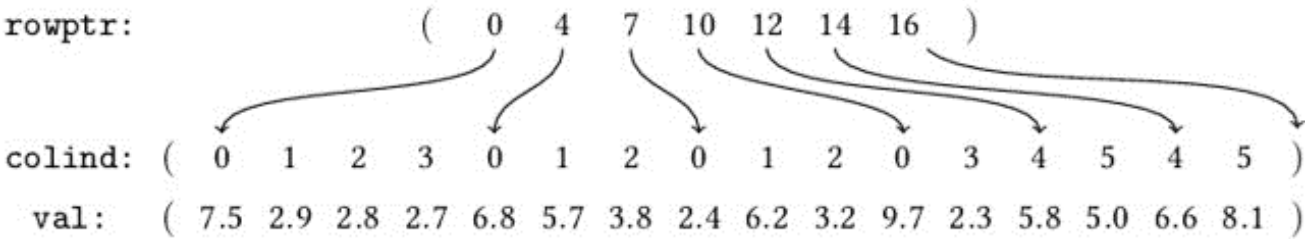
0	0	0	0
5	8	0	0
0	0	3	0
0	4	0	0

has the following **CSR representation**:

- $V = \quad (5\ \ 8\ \ 3\ \ 4)$
- $C_{Index} = (0\ \ 1\ \ 2\ \ 1)$
- $R_{Index} = (0\ \ 0\ \ 2\ \ 3\ \ 4)$

The Compressed Sparse Row (CSR) Representation – Example 2

$$A = \begin{pmatrix} 7.5 & 2.9 & 2.8 & 2.7 & 0 & 0 \\ 6.8 & 5.7 & 3.8 & 0 & 0 & 0 \\ 2.4 & 6.2 & 3.2 & 0 & 0 & 0 \\ 9.7 & 0 & 0 & 2.3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 5.8 & 5.0 \\ 0 & 0 & 0 & 0 & 6.6 & 8.1 \end{pmatrix}$$



Elafrou, A., Goumas, G., & Koziris, N. (2015). A lightweight optimization selection method for Sparse Matrix-Vector Multiplication. arXiv preprint arXiv:1511.02494

The Compressed Sparse Row (CSR) Representation – Exercise

Given the following matrix

0	2	0	0	1	0	0	0
0	8	0	0	0	0	2	0
2	0	3	0	1	1	0	0
0	0	0	0	0	0	8	0

has the following **CSR representation**:

- $|V| = ?$
- $|C_{Index}| = ?$
- $|R_{Index}| = ?$

$V = ?$

$C_{Index} = ?$

$R_{Index} = ?$

Reconstruct the original matrix

The Compressed Sparse Row (CSR) Representation … Cont'd

Advantage:

- CSR is **more compact and is laid out more contiguously in memory** than adjacency lists and adjacency `<vector>`s, eliminating nearly all space overheads and reducing random memory accesses compared with these other formats.

Disdvantage:

- The price we pay for CSR's advantages is **reduced flexibility**: adding new edges to a graph in CSR format is inefficient, so CSR is suitable for graphs whose structure is fixed and given all at once.

From: Compressed Sparse Row Format for Representing Graphs

The Compressed Sparse Column (CSC) Representation

- **Compressed Sparse Column (CSC)** is similar to CSR except that:
  - values are read first by column,
  - a row index is stored for each value,
  - column pointers are stored.

From: Graph Compression – Data compression (Lecture 5)

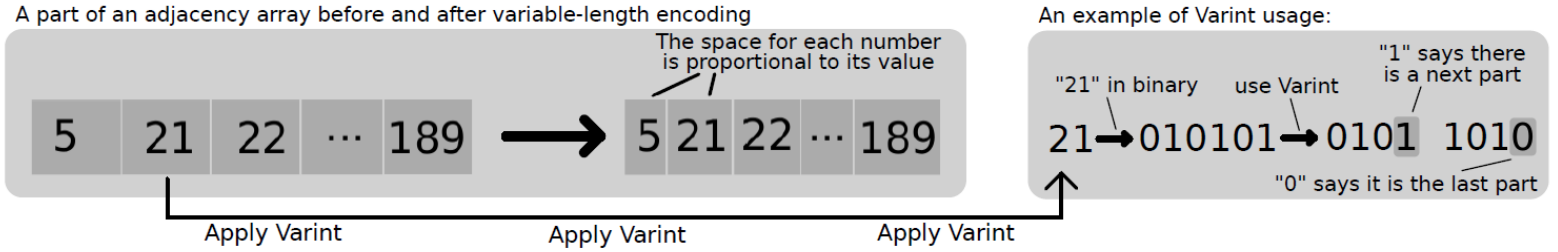
Computational Costs – Time

Operation	Adjacency Matrix	Edge List	Adjacency List	CSR/CSC
scan_graph	$O(n^2)$	$O(m)$	$O(m+n)$	$O(m+n)$
get_neighbors	$O(n)$	$O(m)$	$O(d)$	$O(d)$
is_edge	$O(1)$	$O(m)$	$O(d)$	$O(d)$
insert edge	$O(1)$	$O(1)$	$O(1)$ or $O(d)$	$O(m+n)$
delete edge	$O(1)$	$O(m)$	$O(d)$	$O(m+n)$

From: arXiv preprint arXiv:1806.01799

Variable-Length Encoding

- In **variable-length encoding**, vertex IDs stored in the adjacency array are encoded with one of the selected **variable-length codes***, such as **variable length integer** (varint) coding → details



*In coding theory, a **variable-length code** is a code which maps source symbols to a **variable** number of bits.

From: arXiv preprint arXiv:1806.01799

Huffman Degree Encoding

- The core idea in the **Huffman degree encoding** scheme is to use fewer bits to encode vertex IDs of higher degrees.
  - Vertex IDs that occur more often use fewer bits → saving space.

From: Huffman Encoding and Data Compression
Further info: Huffman Coding

Huffman Degree Encoding … Cont'd

standard ASCII table.

char	ASCII	bit pattern (binary)
h	104	01101000
a	97	01100001
p	112	01110000
y	121	01111001
i	105	01101001
o	111	01101111
space	32	00100000

The string **"happy hip hop"** would be encoded in ASCII as **104 97 112 112 121 32 104 105 112 32 104 111 112**. Although not easily readable by humans, it would be written as the following stream of bits (each byte is boxed to show the boundaries):

01101000	01100001	01110000	01110000	01111001	00100000	01101000
01101001	01110000	00100000	01101000	01101111	01110000	

From: Huffman Encoding and Data Compression
Further info: Huffman Coding

Huffman Degree Encoding … Cont'd

optimal Huffman encoding for the string **"happy hip hop"**:

char	bit pattern
h	01
a	000
p	10
y	1111
i	001
o	1110
space	110

Each character has a unique bit pattern encoding, but not all characters use the same number of bits. The string **"happy hip hop"** encoded using the above variable-length code table is:

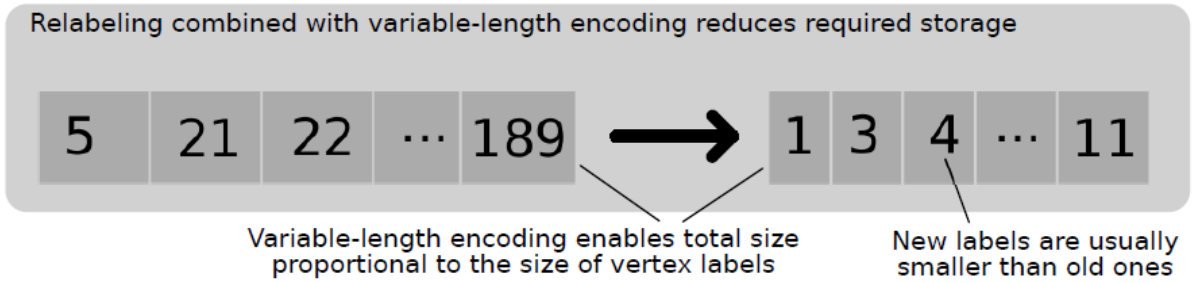
01	000	10	10	1111	110	01	001	10	110	01	1110	10

The encoded phrase requires a total of 34 bits, shaving a few more bits from the fixed-length version.

From: arXiv preprint arXiv:1806.01799

Vertex Relabeling

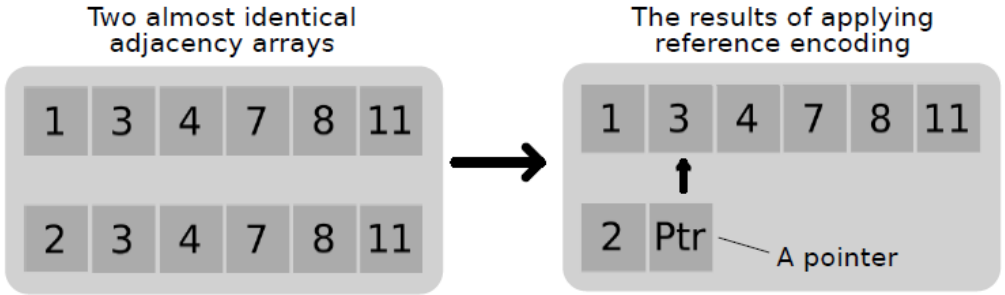
- In **vertex relabeling**, the main idea is to change the initial IDs of vertices so that the new IDs, when stored, use less space. We also use the name **vertex permutations** to refer to this technique.
- This scheme is usually combined with variable-length encoding.



From: arXiv preprint arXiv:1806.01799

Reference Encoding

- In **reference encoding**, identical sequences of vertices in the adjacency arrays of different vertices are identified.
- Then, all such sequences (except for a selected one) are encoded with references.



From: arXiv preprint arXiv:1806.01799

Gap Encoding

- The **gap encoding** scheme preserves differences between vertex IDs rather than the IDs themselves.
  - The motivation is that, in most cases, differences occupy less space than IDs.

- **Several variants** can be used:
  - The most popular is storing differences between the IDs of the consecutive neighbors of each vertex v, for example:
    $$N_1(v) - v, N_2(v) - N_1(v), \dots, N_{d_v-1}(v) - N_{d_v-2}(v), N_{d_v}(v) - N_{d_v-1}(v)$$
    (the first of the above differences is sometimes called an initial distance and each following: an increment).
  - Another variant stores the differences between v and each of its neighbors:
    $$N_1(v) - v, N_2(v) - v, \dots, N_{d_v-1}(v) - v, N_{d_v}(v) - v$$

Gap Encoding … Cont'd

- **Original representation**:

1	→	3	5	8	44	88	120

- **Gap encoding**:

1	→	2	2	3	36	44	32

Gap Encoding … Exercise

- **Original representation**:

1	→	43	55	88	464	868	1250

- **Gap encoding**:

1	→	?	?	?	?	?	?

From: arXiv preprint arXiv:1806.01799

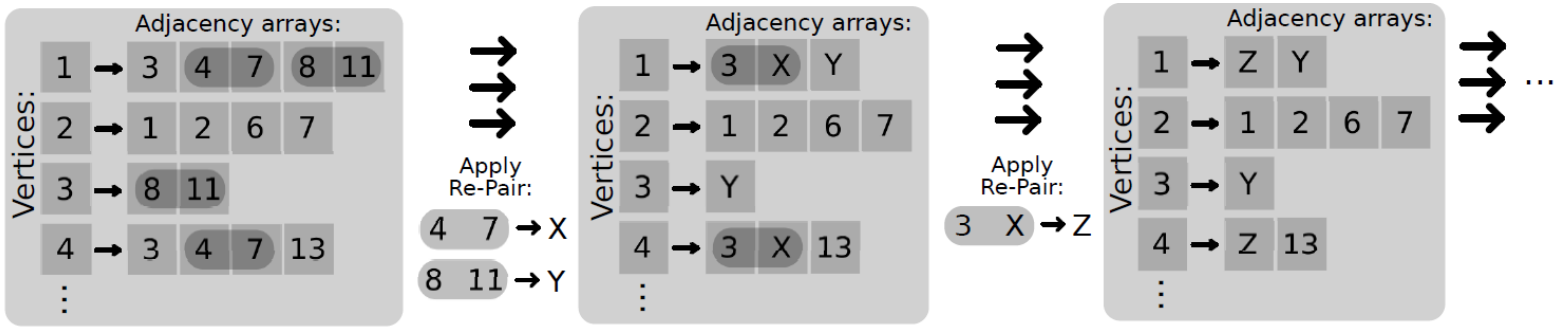
Re-Pair (Claude and Navarro, 2007)

- In the context of Web Graph compression, the authors propose a **text-based approach for graph compression**.
- A **phrase-based compression scheme** that enables fast decompression that is also local:
  - It does not always require accessing the whole graph.
- Re-Pair repeatedly **finds the most frequent pairs of symbols in a given graph representation** and replaces them with new symbols.
- This is repeated as long as storage is reduced.

Claude, F., & Navarro, G. (2007, October). A fast and compact Web graph representation. In International Symposium on String Processing and Information Retrieval (pp. 118-129). Springer, Berlin, Heidelberg

From: arXiv preprint arXiv:1806.01799

Re-Pair (Claude and Navarro, 2007) … Cont'd



From: arXiv preprint arXiv:1806.01799

k^2 Trees (Brisaboa et al., 2014)

- A graph representation model where **a graph is modeled with a tree**.
- Initially, the graph is divided into k^2 submatrices of identical size (k is a parameter); these submatrices are recursively divided in the same way.
- Now, the key idea is to represent the graph as a k^2-ary tree (called a k^2 tree) that corresponds to the above recursive "partitioning" of the graph.
- At every partitioning level, if a given submatrix to be partitioned contains only 0s, the corresponding tree node contains 0. Otherwise, it contains a 1.

Brisaboa, N. R., Ladra, S., & Navarro, G. (2014). Compact representation of Web graphs with extended functionality. Information Systems, 39, 152-174

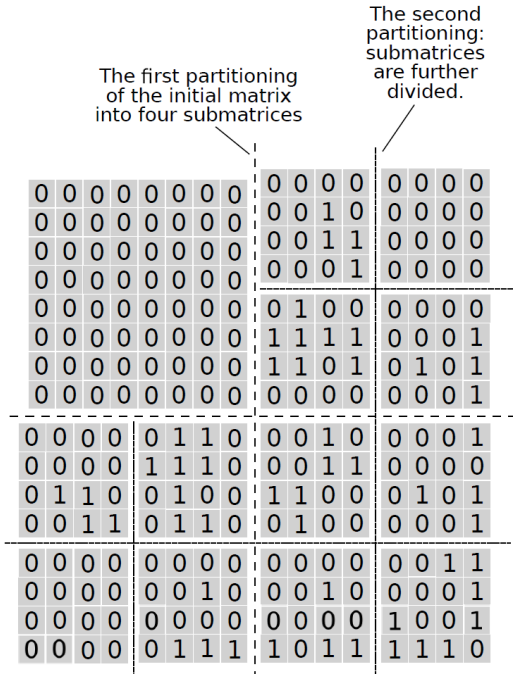
From: arXiv preprint arXiv:1806.01799

k^2 Trees (Brisaboa et al., 2014) ... Cont'd

An adjacency matrix of some graph:

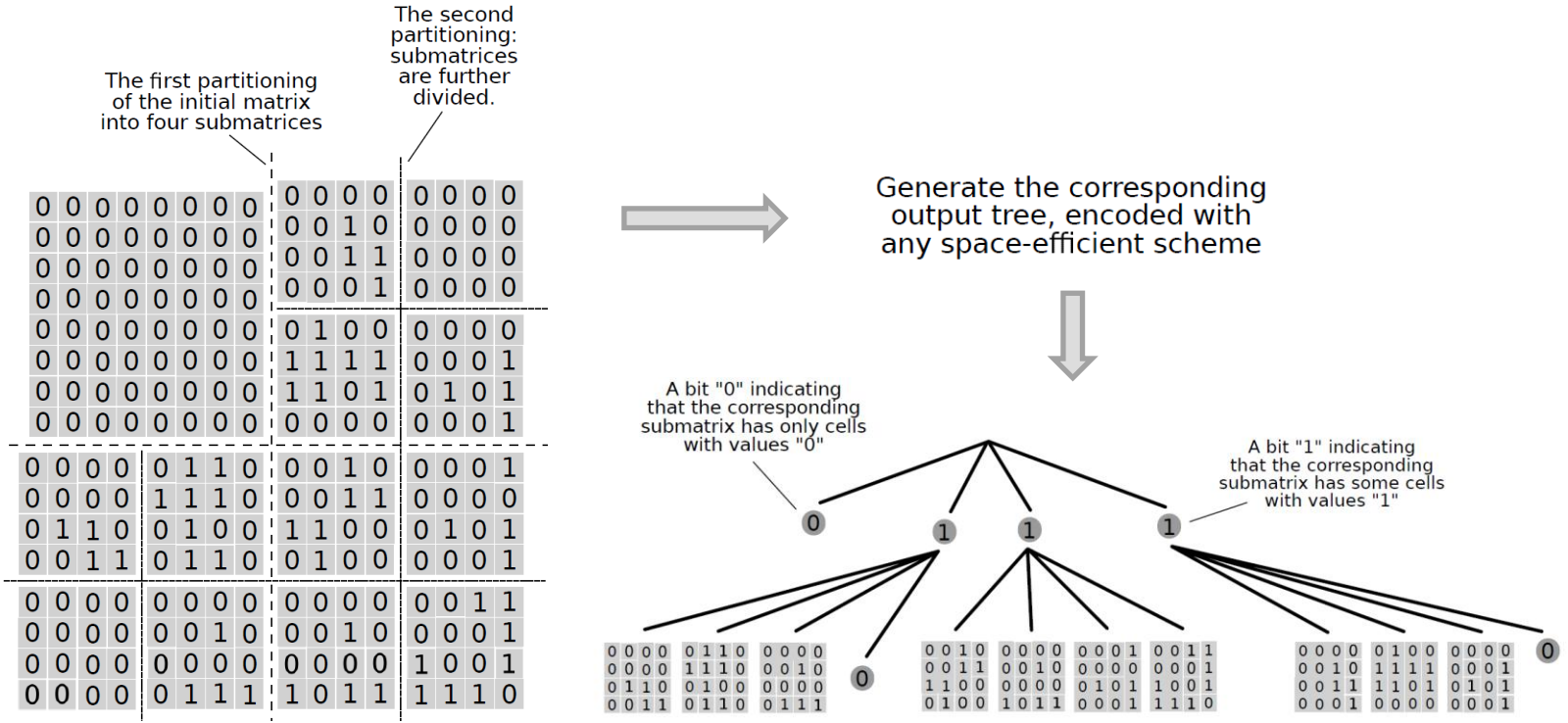
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	1
0	0	0	0	0	0	0	0	1	1	0	1	0	1	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	1	1	0	0	0	1	0	0	0	0	1
0	0	0	0	1	1	1	0	0	0	1	1	0	0	0	0
0	1	1	0	0	1	0	0	1	1	0	0	0	1	0	1
0	0	1	1	0	1	1	0	0	1	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1
0	0	0	0	0	1	1	1	1	0	1	1	1	1	1	0

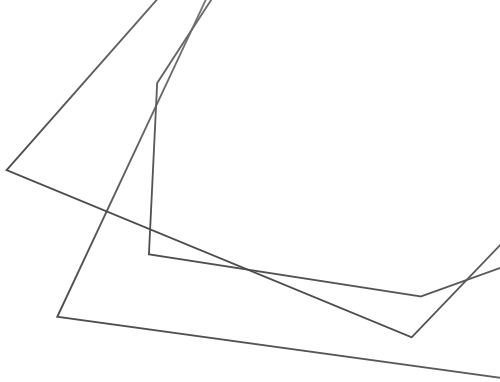
Partition the matrix for k=2, using two partitoning levels (more levels could compress the matrix better; we use two for simplicity of the example)



From: arXiv preprint arXiv:1806.01799

k^2 Trees (Brisaboa et al., 2014) … Cont'd

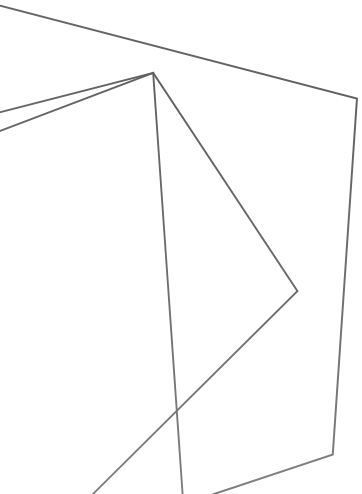




3

Graph Summarization

Intro and Taxonomy of Approaches



From: https://arxiv.org/abs/1612.04883

Definition and relationship with Compression

- **Graph summarization** methods leverage compression to find a smaller representation of the input graph, while discovering **structural patterns**.

- Graph summarization and compression are related.
  - In graph summarization, although compression is the means, finding the absolutely smallest representation of the graph is not the end goal.
    - The patterns that are being unearthed during the process may lead to suboptimal compression.
  - In graph compression, the goal is to compress the input graph as much as possible to minimize storage space, irrespective of patterns.

From: https://arxiv.org/abs/1612.04883

Benefits

- **Reduction**: volume and storage.
- **Speedup**: graph algorithms & queries.
- **Interactive analysis**
- **Noise elimination**

From: https://arxiv.org/abs/1612.04883

Applications

- **Clustering, Classification, Community detection**
- **Model order selection in matrix factorization**
- **Outlier detection, pattern set mining**
- **Finding sources of infection in large graphs**
- **Understanding selected nodes in graphs**

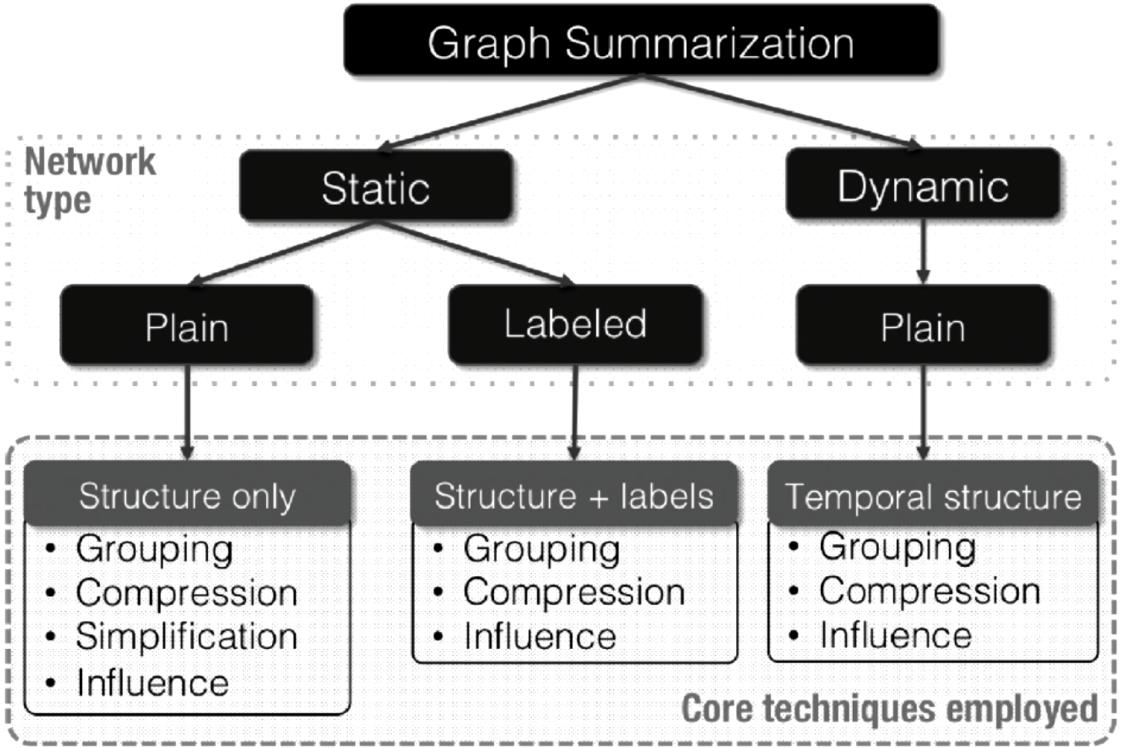
From: https://arxiv.org/abs/1612.04883

Challenges

- **Data volume**
- **Complexity of data**
- **Definition of «interestingness»**
- **Evaluation**
- **Change over time**

From: https://arxiv.org/abs/1612.04883

Graph Summarization Algorithms



From: https://arxiv.org/abs/1612.04883

Types of Graph Summaries

A **possible taxonomy** of graph summarization approaches can be divided into:

- Methods based on the **input data** handled.
- Methods based on the **core techniques** employed.

From: https://arxiv.org/abs/1612.04883

Input: Static or Dynamic

- Most summarization methods operate on **static networks**, leveraging graph structure (links), and, if available, the node/edge attributes.
- Despite the prevalence of large dynamic networks, only recent research efforts address their efficient summarization.
- In some cases, static methods are adapted to handle dynamic networks seen as series of static snapshots.

From: https://arxiv.org/abs/1612.04883

Input: Homogeneous or Heterogeneous

- The most well-studied instance in graph summarization, and graph mining more generally, is the **homogeneous graph** with one entity and one link type.

- However, some approaches apply to heterogeneous graphs by treating various types of nodes (e.g., students, instructors) and relations between them (e.g., teacher, friends, classmates) differently.

From: https://arxiv.org/abs/1612.04883

Core Technique: Grouping- or Aggregation-based

- This is the **most popular technique**.
- Some **node-grouping methods** recursively aggregate nodes into "supernodes" based on an application-dependent optimization function, which can be based on structure and/or attributes.
  - Others employ existing clustering techniques and map each densely connected cluster to a supernode.
- **Edge-grouping methods** aggregate edges into compressor or virtual nodes.

From: https://arxiv.org/abs/1612.04883

Core Technique: Bit-compression-based

- This approach, a **common technique in data summarization**, minimizes the number of bits needed to describe the input graph via its summary.
- Some methods are **lossless** and can perfectly reconstruct the original graph from the summary.
- Others are **lossy**, compromising recovery accuracy for space savings.

From: https://arxiv.org/abs/1612.04883

Core Technique: Simplification-based

- These methods streamline an input graph by **removing less "important" nodes or edges**, resulting in a simplified (or sparsified) graph.
- How to consider the concept of "impotance"?
  - Domain-/application-dependent.

From: https://arxiv.org/abs/1612.04883

Core Technique: Influence-based

- These approaches aim to discover a **high-level description of the influence propagation** in large-scale graphs.
- Techniques in this category formulate the summarization problem as an optimization process in which some quantity related to information influence is maintained.

From: https://arxiv.org/abs/1612.04883

What about the Output?

Output: Summary type

- A **supergraph**, which consists of supernodes or collections of original nodes, and superedges between them.
- A **simplified/sparsified graph**, which has fewer nodes and/or edges than the original network.
- A **list of (static or temporal) structures** or **influence propagations**, which are seen independently instead of in the form of a single summary graph.

From: https://arxiv.org/abs/1612.04883

What about the Output? … Cont'd

Output: Summary type

- **Flat**, with nodes simply grouped into supernodes.
- **Hierarchical**, with multiple levels of abstraction.

From: https://arxiv.org/abs/1612.04883

What about the Output? … Cont'd

Output: Non-overlapping or overlapping nodes

- In its simplest form, a summary is **non-overlapping**: each original node belongs only to one summary element (e.g., supernode, subgraph).
- Conversely, **overlapping** summaries, where a node may belong to multiple elements, can capture complex inherent data relationships but may also complicate interpretation and visualization.

From: https://arxiv.org/abs/1612.04883

Grouping-Based Methods

- **Grouping-based (aggragation-based) methods** are among the most popular techniques for summarization.

- We distinguish grouping-based graph summarization methods into two main categories:
  - Node-grouping,
  - Edge-grouping.

From: https://arxiv.org/abs/1612.04883

Node-Grouping Methods

- Some approaches **recursively aggregate nodes** into supernodes, connected via superedges, based on an application-dependent optimization function.
- Others employ existing **clustering techniques** to find clusters that then map to supernodes.

From: https://arxiv.org/abs/1612.04883

Node Clustering-based Methods

- Although **the goal of clustering is not graph summarization**, the outputs of clustering algorithms can be easily converted to non-application-specific summaries.

- A small representation of the input graph can be obtained by
  - Mapping all the nodes that belong to the same cluster/community to a supernode.
  - Linking them with superedges with weight equal to the sum of the cross-cluster edges or else the sum of the weights of the original edges.

From: https://arxiv.org/abs/1612.04883

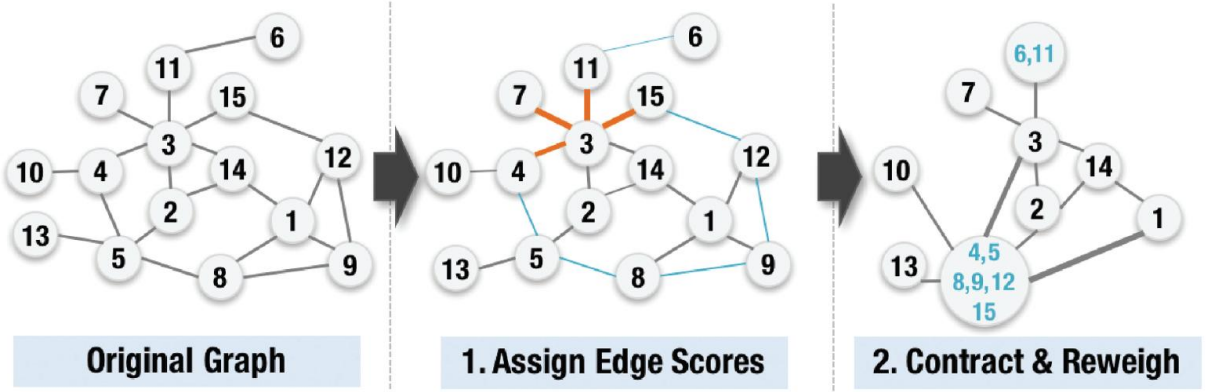
Node Clustering-based Methods … Cont'd

- Although the clustering output can be viewed as a summary graph, a **fundamental difference** from tailored summarization techniques is that:
  - Summarization groups nodes that are linked to the rest of the graph in a similar way.
  - Clustering methods simply group densely connected nodes.

From: https://arxiv.org/abs/1612.04883

Node Aggregation-based Methods

- **Aggregation-based methods** focus on the aggregation of the vertices of the graph. This aggregation is done using several strategies.



From: https://arxiv.org/abs/1612.04883

Node Aggregation-based Methods … Cont'd

- Some methods aims at **minimizing some version of the approximation or reconstruction error**.
  - GraSS (LeFevre and Terzi, 2010)
    - Normalized reconstructed error

$$E = \frac{1}{|V|^2} \sum_{i \in V} \sum_{j \in V} |\bar{A}(i,j) - A(i,j)|$$

  - Riondato et al. (2017)
    - l_p reconstruction error
    - The p-norm of

$$A - \bar{A}$$



Further info about norms

LeFevre, K., & Terzi, E. (2010, April). GraSS: Graph structure summarization. In Proceedings of the 2010 SIAM International Conference on Data Mining (pp. 454-465). Society for Industrial and Applied Mathematics

Riondato, M., García-Soriano, D., & Bonchi, F. (2017). Graph summarization with quality guarantees. Data mining and knowledge discovery, 31(2), 314-349

From: https://arxiv.org/abs/1612.04883

Edge-Grouping Methods

- **Edge-grouping methods** aggregate edges into compressor or virtual nodes to reduce the number of edges in a graph in either a lossless or lossy way.

- Note that in this section, **“compression” does not refer to bit-level optimization**, as in the following section but rather to the process of replacing a set of edges with a node.

From: https://arxiv.org/abs/1612.04883

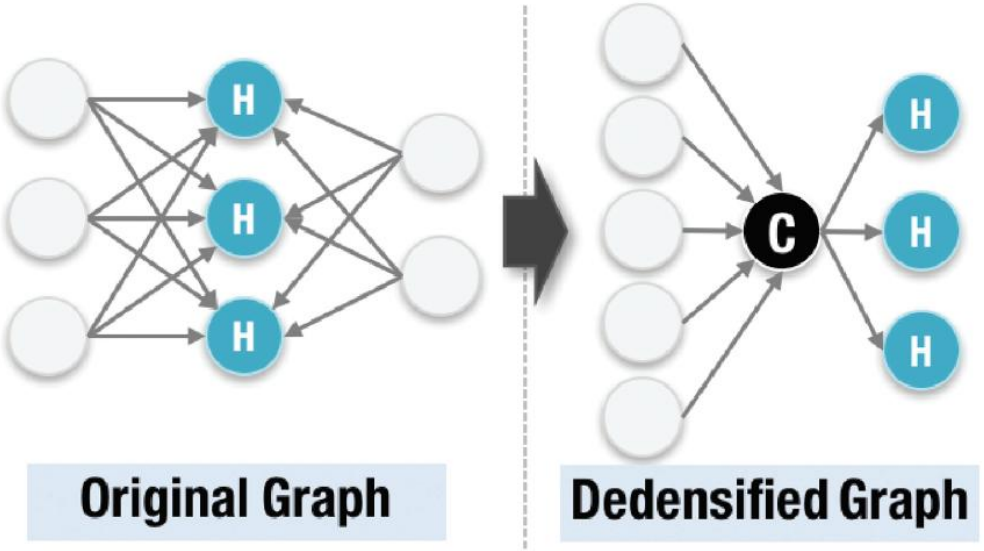
Graph Dedensification (Maccioni and Abadi 2016)

- An edge-grouping method that **compresses neighborhoods around high-degree nodes**.
- Following the **assumption that high-degree nodes are surrounded by redundant information** that can be synthesized and eliminated, the authors introduce “compressor nodes”, which represent common connections high-degree nodes.
- **Dedensification** only occurs when every node has at most one outgoing edge to a compressor node, and every high-degree node has incoming edges coming only from a compressor node.

Maccioni, A., & Abadi, D. J. (2016, August). Scalable pattern matching over compressed graphs via dedensification. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 1755-1764)

From: https://arxiv.org/abs/1612.04883

Graph Dedensification (Maccioni and Abadi 2016) … Cont'd



From: https://arxiv.org/abs/1612.04883

Bit Compression-Based Methods

- **Bit compression** is a common technique in data mining.
- In graph summarization, the goal of these approaches is to **minimize the number of bits needed to describe the input graph**, where the summary consists of a model for the input graph and its unmodeled parts.
- The graph summary or model is significantly smaller than the original graph and often reveals various structural patterns, like bipartite subgraphs, that enhance understanding of the original graph structure.

From: https://arxiv.org/abs/1612.04883

Bit Compression-Based Methods … Cont'd

- As previously mentioned, **some of these approaches primarily use compression** and **secondary grouping techniques**.
- However, some **others aim solely to compress a given graph** without necessarily creating a graph summary or finding comprehensible graph structures → Graph compression!

From: https://arxiv.org/abs/1612.04883

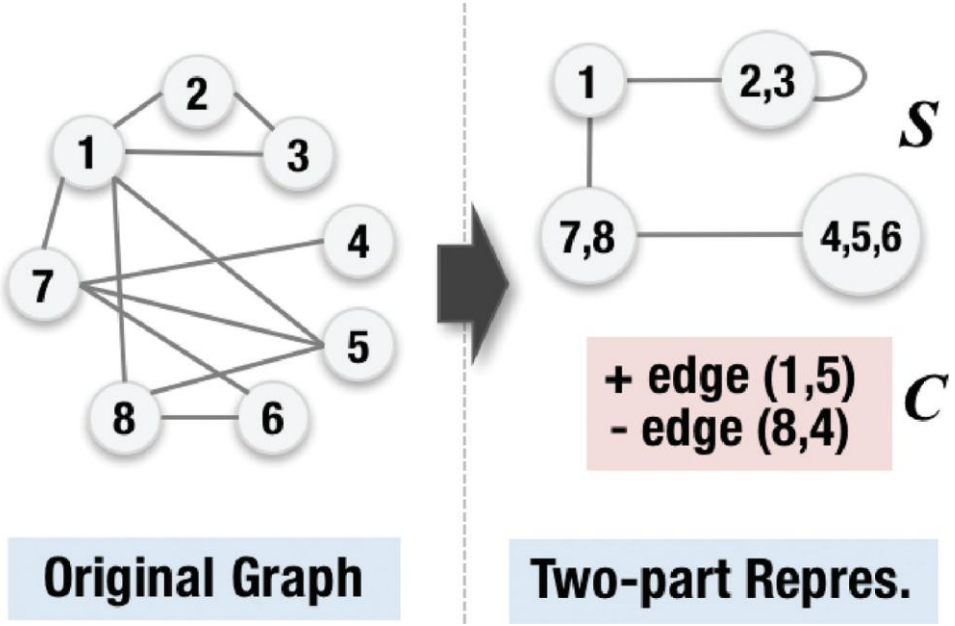
Two-part MDL (Navlakha et al., 2008)

- This representation, obtained by **aggregating nodes in the summary generation**, consists of a graph summary S and a set of corrections C.
- The summary is an aggregate graph in which each node corresponds to a set of nodes in G, and each edge represents the edges between all pairs of nodes in the two sets.
- The correction term specifies the list of edge-corrections that must be applied to the summary to exactly recreate G.

Navlakha, S., Rastogi, R., & Shrivastava, N. (2008, June). Graph summarization with bounded error. In Proceedings of the 2008 ACM SIGMOD international conference on Management of data (pp. 419-432)

From: https://arxiv.org/abs/1612.04883

Two-part MDL (Navlakha et al., 2008) … Cont'd

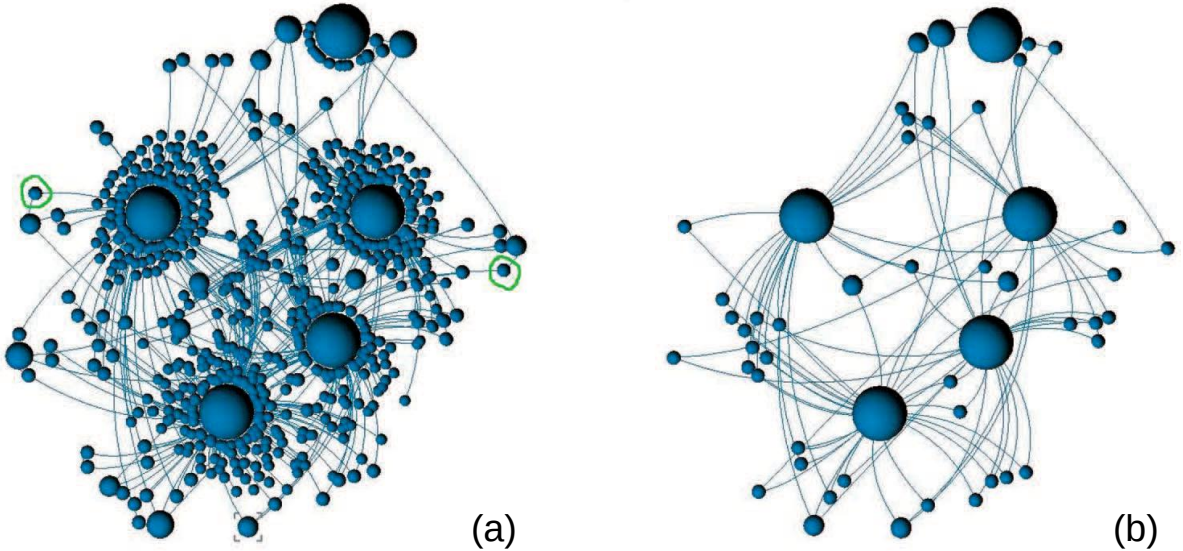


From: https://arxiv.org/abs/1612.04883

Simplification-Based Methods

- **Simplification-based summarization** methods streamline the original graph by **removing less "important" nodes or edges**, resulting in a simplified (or sparsified) graph.
- As opposed to supergraphs, here the summary graph consists of a **subset of the original nodes and/or edges**.
- In addition to simplification-based summarization methods, some existing graph algorithms have the potential for simplification-based summarization, such as:
  - Sparsification
  - Sampling
  - Sketching

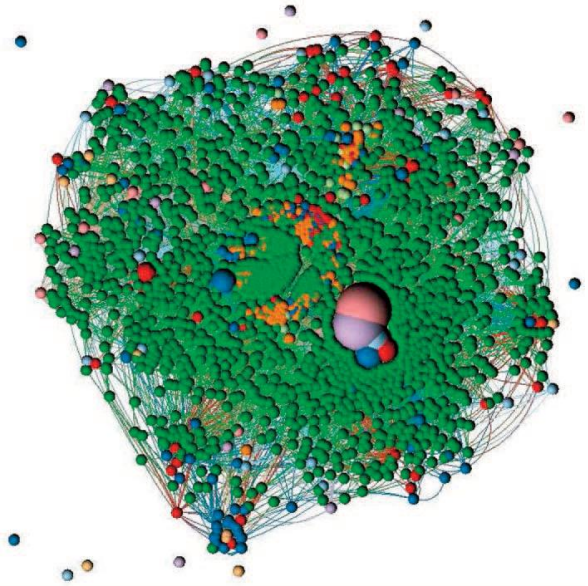
OntoVis (Shen et al., 2006)



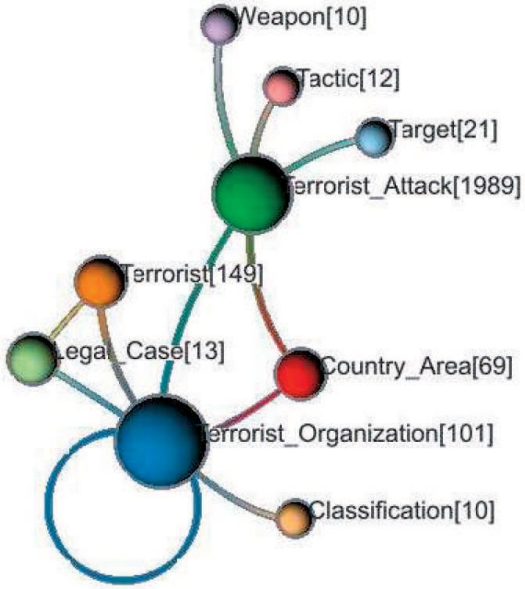
- (a) Original network has many one-degree nodes and duplicate paths
- (b) Structural abstraction after removing one-degree nodes and duplicate paths.

Shen, Z. et al. (2006). Visual analysis of large heterogeneous social networks by semantic and structural abstraction. IEEE transactions on visualization and computer graphics, 12(6), 1427-1439.

OntoVis (Shen et al., 2006) … Cont'd



Visualization of the Entire Terrorism Network. There are 2,374 nodes and 8,767 links.



Ontology Graph of the Terrorism Network. There are nine node types including terrorist organization, classification, terrorist, legal case, country/area, attack, attack target, weapon, and tactic.

From: https://arxiv.org/abs/1612.04883

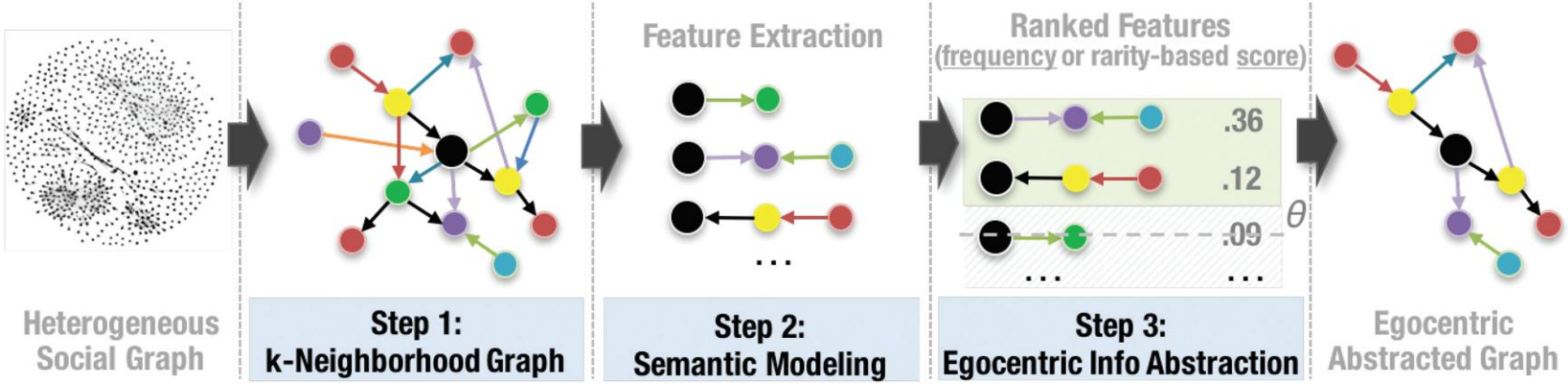
Egocentric Abstraction (Li and Lin, 2009)

- A **four-step unsupervised algorithm for egocentric information abstraction of heterogeneous social networks** using edge filtering (instead of node filtering)
  1. During the semantic modeling step, features (or else linear combinations of relations or path-based patterns) are automatically selected and extracted according to the surrounding network substructure (k-hop neighborhoods).
  2. The statistical dependency is measured between the features per ego node.
  3. During the egocentric information abstraction step, irrelevant information is removed by applying distilling criteria, such as keeping the most frequent or rare features.
  4. An egocentric abstracted graph is constructed incrementally on the remaining features, allowing the user to visualize the smaller resulting graph.

Li, C. T., & Lin, S. D. (2009, July). Egocentric information abstraction for heterogeneous social networks. In 2009 International Conference on Advances in Social Network Analysis and Mining (pp. 255-260). IEEE

From: https://arxiv.org/abs/1612.04883

Egocentric Abstraction (Li and Lin, 2009) … Cont'd



From: https://arxiv.org/abs/1612.04883

Influence-Based Methods

- **Influence-based methods** seek to find a compact, high-level description of the influence dynamics in large-scale graphs to understand the patterns of influence propagation at a global level.
- Usually, such methods formulate graph summarization as an **optimization process** in which some quantity related to information influence is maintained.
- These summarization methods are scarce and have been mostly applied on social graphs, where important influence-related questions arise.

From: https://arxiv.org/abs/1612.04883

Community-level Social Influence (CSI) (Mehmood et al. 2013)

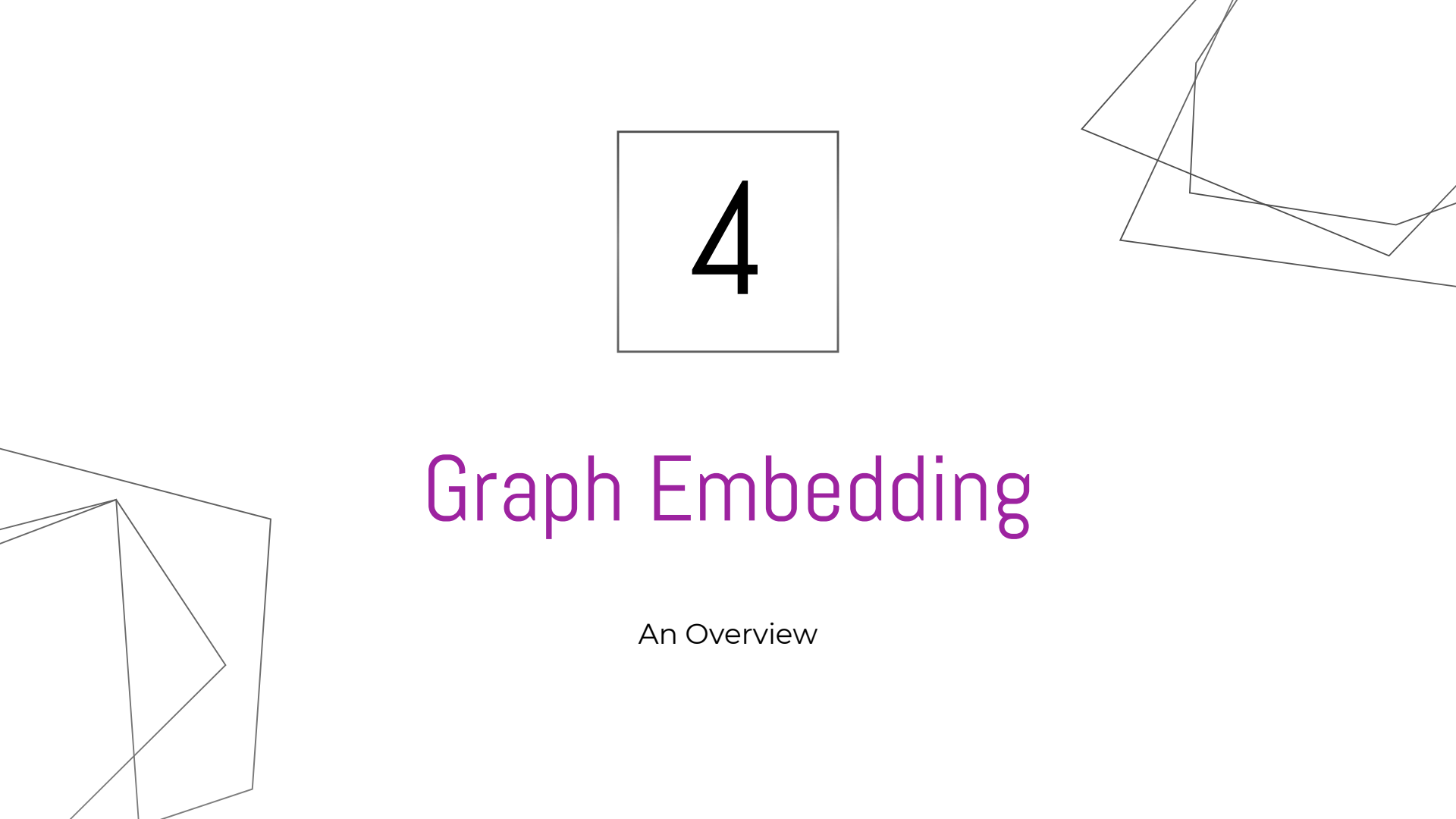
- **CSI focuses on summarizing social networks** via **information propagation** and **social influence analysis**.
- CSI **relies on existing clustering approaches**: it detects a set of communities using METIS (Karypis and Kumar 1999) and then finds their reciprocal influence by extending the Independent Cascade Model to communities instead of individual nodes.

From: https://arxiv.org/abs/1612.04883

Community-level Social Influence (CSI) … Cont'd

- Unlike other influence propagation approaches that find representative cascades for information diffusion, **CSI leads to a compact representation of the input network** where the nodes correspond to communities and the directed edges represent influence relationships.

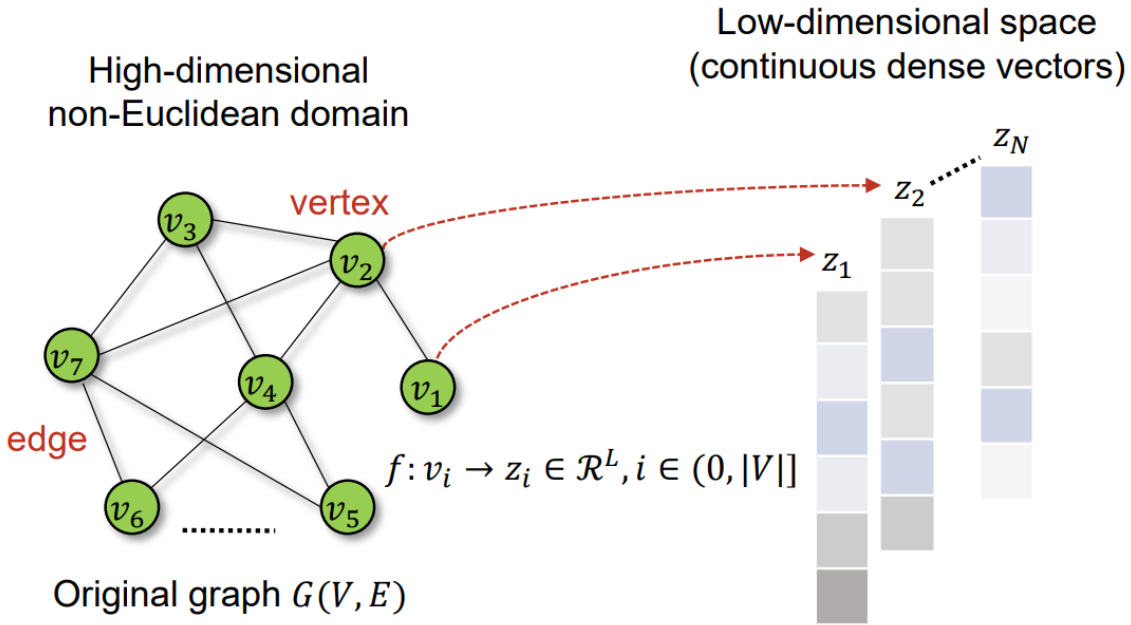
- The **output of CSI** is different from grouping-based summarization techniques in which the superedges simply represent aggregate connections between the adjacent supernodes.



Graph Embedding

An Overview

A Graphical Representation



Source: https://doi.org/10.1137/20M1386062

A Taxonomy of Approaches

- A **broad taxonomy** of graph embedding methods used in the literature points to three major categories:
    - Matrix factorization-based methods
    - Random walk-based methods
    - Deep learning-based methods

Matrix Factorization-Based Methods

- Matrix factorization-based methods construct a **high-order proximity matrix** based on graph properties (e.g., transition probabilities, node pairwise similarity, etc.) and factorize it to obtain the node embeddings.
    - Node adjacency matrix, Laplacian matrix, node transition probability matrix, and Katz similarity matrix, among others.
- They cannot easily scale up to large network embedding.

Goyal, P., & Ferrara, E. (2018). Graph embedding techniques, applications, and performance: A survey. *Knowledge-Based Systems*, *151*, 78-94

Random Walk-Based Methods

- Embedding techniques using **random walks on graphs** to obtain node representations have been proposed
    - DeepWalk and node2vec are two popular examples.

Perozzi, B., Al-Rfou, R., & Skiena, S. (2014, August). Deepwalk: Online learning of social representations. In Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining (pp. 701-710)

Grover, A., & Leskovec, J. (2016, August). node2vec: Scalable feature learning for networks. In Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining (pp. 855-864)

DeepWalk: Overview

- DeepWalk is an algorithm used to **learn latent representations** (embeddings) of **nodes** in a graph.
- DeepWalk generates **short random walks** over the graph and treats the sequence of nodes visited in each walk as **sentences**.
- These "sentences" are then input into the **Skip-gram** model (from word2vec) to learn vector representations of the nodes.

DeepWalk: Step by Step

Random Walk Generation:

- For each node in the graph, perform several truncated random walks (of fixed length).
- These walks are essentially sequences of node IDs, analogous to word sequences.

Context Window:

- Define a window size (like in word2vec).
- For each node in a walk, the nodes within its window are treated as context nodes.

Training Skip-gram:

- Skip-gram is used to maximize the probability of predicting a node's context given the node.

Output:

- Each node is mapped to a dense, low-dimensional vector (embedding) that captures the structural properties of the graph.

Deep Learning-Based Methods

- Recently, **SDNE** and **DNGR** utilized deep autoencoders to generate an embedding model that can capture nonlinearity in graphs.
    - Structural Deep Network Embedding (SDNE)
    - Deep Neural networks for learning Graph Representations (DNGR)

Wang, D., Cui, P., & Zhu, W. (2016, August). Structural deep network embedding. In Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining (pp. 1225-1234)

Cao, S., Lu, W., & Xu, Q. (2016, February). Deep neural networks for learning graph representations. In Proceedings of the AAAI conference on artificial intelligence (Vol. 30, No. 1)

Possible Assignments

- Deepen a graph summarization approach that is based on the concept of maintaining the dynamics of influence within the graph.
- Critically discuss various models for evaluating the maintenance of influence within graphs.
- Illustrate in detail one of the factorization-, random walk-, or deep learning-based methods for graph embedding.